基于 YOLO11n 的桌面物体检测与 ROS2 通信实验

李书宇
中国海洋大学信息科学与工程学部计算机学院
2024 级计算机科学与技术（中外合作办学）
学号：24020036035
课程：机器人集成小组项目　指导教师：周小伟

摘　要

本实验面向桌面场景中的物体识别任务，使用 YOLO11n 检测鼠标、键盘、笔记本电脑、杯子和头戴式耳机，并将检测结果接入 ROS2。训练数据共 5000 张，含 8507 个目标框。其中 1000 张为自主采集来源素材，另 4000 张取自 Open Images V7。数据划分时以母图为分组单位，随后检查标签与图片的对应关系。模型在 640 像素输入下训练 12 轮。650 张测试图像上的 Precision 为 0.8385，Recall 为 0.7697，mAP@0.5 和 mAP@0.5:0.95 分别为 0.8386 和 0.5735。在另外保存的 20 张单目标图片中，19 张得到正确识别，正确率为 95%。Apple M4 MPS 环境下的视频测试共计时 130 帧，平均端到端速度为 60.409 FPS。程序支持图片、视频和摄像头三种输入，可保存检测记录与错误样本，并通过 ROS2 发布检测字段和带框图像。

关键词：目标检测；YOLO11n；桌面物体；ROS2；边缘视觉；迁移学习

***Abstract*—This experiment investigates five-class object detection in desktop scenes and connects the detector to ROS2. The classes are mouse, keyboard, laptop, cup, and headphones. The dataset contains 5,000 images and 8,507 bounding boxes, including 1,000 samples derived from self-collected photographs and 4,000 images selected from Open Images V7. Images originating from the same parent photograph were kept in one data split, and image–label pairs were checked before training. YOLO11n was fine-tuned for 12 epochs with an input size of 640 pixels. Evaluation on 650 test images produced a precision of 0.8385, a recall of 0.7697, an mAP at IoU 0.5 of 0.8386, and an mAP averaged over IoU 0.5–0.95 of 0.5735. The model also identified 19 of 20 stored single-object samples correctly. In a 130-frame video test on Apple M4 MPS, the complete processing loop averaged 60.409 FPS. The program accepts images, videos, or camera streams, records detections and error cases, and publishes both structured results and annotated images as ROS2 topics.**

***Index Terms*—object detection, YOLO11n, desktop objects, ROS2, edge vision, transfer learning**

I. 引言

在桌面抓取或人机交互任务中，机器人需要知道物体的类别和位置，单纯给出图像级分类结果并不够。实际拍摄时，鼠标、键盘和笔记本电脑往往相互靠近，打开的笔记本还同时包含整机区域与可见键盘区域。光照、遮挡和拍摄角度也会改变目标外观。因此，识别程序除了输出类别，还应给出边界框与置信度，并能连续处理摄像头画面。

YOLO 在一次前向传播中同时完成类别预测和边界框回归，属于单阶段检测方法 [1]。SSD 则利用不同尺度的特征图处理尺寸差异较大的目标 [2]。这类方法省去了候选区域生成步骤，推理链路较短，更适合接入摄像头程序。本实验据此选用参数量较小的 YOLO11n，在五类桌面物体数据上继续训练，并通过 ROS2 发布检测结果。

实验工作覆盖数据整理、模型训练、结果测试和 ROS2 节点实现。评价部分既统计测试集上的逐类指标，也使用 20 张单目标图片核对课程要求，并在固定视频上记录完整处理速度。模型结构本身没有改动，工作的重点是把数据、推理程序、结果保存和通信接口连成一条可运行的实验流程。

II. 相关工作

A. 单阶段目标检测

原始 YOLO 直接从整幅图像预测多个边界框及其类别概率，省去了区域提议与二次分类 [1]。SSD 在多层特征图上设置默认框，使不同尺寸的目标可以在相应分辨率上被检测 [2]。对于单阶段检测器，前景与背景样本的不平衡会影响训练。Focal Loss 通过降低易分类样本的损失权重，使网络更多关注困难样本 [3]。这些工作分别从推理速度、多尺度预测和样本权重三个角度改进了稠密检测。

数据增强只作用于训练阶段，不增加部署时的网络计算量。YOLOv4 将颜色变化、几何变换和 Mosaic 等方法组合用于检测训练，并比较了实时场景下的速度与精度 [4]。考虑到桌面物体通常不会发生大幅形变，本实验只设置小角度旋转，并配合平移、缩放、水平翻转和概率为 0.5 的 Mosaic。

YOLO11 由 Ultralytics 软件包提供，训练、验证、预测和模型导出均使用同一套接口，其中 n 规模模型用于轻量推理 [5]。目前没有单独介绍 YOLO11 架构的正式论文，这里以官方代码标明所用模型版本。早期 YOLO 文献仅用于说明单阶段检测思路。

B. 公开数据、评价指标与机器人通信

Open Images 收录了自然场景图像及多种标注，目标检测子集包含大量多目标画面 [6]。本实验从其 V7 版本中筛选五个对应类别，再把边界框转换为 YOLO 格式。评价时同时报告 Precision、Recall、mAP@0.5 和 mAP@0.5:0.95。后一个指标沿用 COCO 的多 IoU 阈值评价方式 [7]，具体数值由 Ultralytics 验证接口计算 [8]。

ROS2 使用 topic 传递传感器数据和机器人状态。发布者与订阅者只需约定主题名称和消息类型，不必直接调用对方程序 [9]。因此，目标检测可以单独运行成一个节点：节点读取图像并完成推理，其余程序只订阅所需的数值结果或可视化图像。

TABLE I
数据集划分与目标框统计

划分	图像	标签	空标签	目标框
Train	4000	4000	0	6773
Validation	350	350	0	627
Test	650	650	0	1107
合计	5000	5000	0	8507

TABLE II
五类目标框分布

类别	目标框数	比例（%）
mouse	1724	20.27
keyboard	1966	23.11
laptop	2549	29.96
cup	1397	16.42
headphones	871	10.24
合计	8507	100.00

III. 系统设计与方法

A. 总体架构

程序流程见图1。数据整理脚本负责公开数据筛选、标签转换和数据划分。训练脚本读取 YAML 配置并生成模型权重。推理程序与训练入口分开，可接收图片、视频或 USB 摄像头画面，输出逐帧日志、带框结果和错误样本。ROS2 节点复用同一推理配置，再把结果封装为消息。这样更换输入源时无需修改训练代码。

一条检测记录包含类别编号、类别名称、置信度和像素坐标边界框 (x_1, y_1, x_2, y_2)。输入图像缩放到 640×640 后送入网络，候选框先经过置信度筛选，再用非极大值抑制去除重复框。保留下来的结果一份用于绘制类别和边界框，另一份写入 CSV。视频与图片可按运行参数保存。

B. 模型训练策略

训练从已有的五类模型权重开始，而不是随机初始化。重新整理的数据用于继续微调 YOLO11n。正式训练运行 12 轮，batch size 设为 32，输入尺寸为 640，随机种子为 0，并开启确定性选项。优化器由 Ultralytics 自动选择。初始学习率参数、动量和权重衰减分别设为 0.01、0.937 和 0.0005。

训练时在线加入颜色和几何变化。色相、饱和度和明度参数分别为 0.015、0.7 和 0.4。旋转范围为 5 度，平移与缩放比例为 0.1 和 0.3，水平翻转概率为 0.5。Mosaic 概率设为 0.5，MixUp 关闭。训练结束后，根据验证集指标保存最佳权重，同时保留最后一轮权重。

IV. 数据集构建

A. 类别体系与标注

五个类别按 mouse、keyboard、laptop、cup、headphones 的顺序编号为 0 至 4。标签采用 YOLO 水平边界框格式，每行记录类别编号、归一化中心坐标以及归一化宽高。遇到打开的笔记本电脑时，整机标为 laptop，可见键盘区域另标为 keyboard。被遮挡物体只框选可见部分。

数据划分见表I。5000 张图像均有对应标签文件，共标注 8507 个目标框。训练集含 4000 张图像，验证集含 350 张，余下 650 张作为测试集。最佳权重按验证结果选取，测试集不参与训练过程。

B. 数据来源与划分控制

数据集中，自主采集来源素材为 1000 张，Open Images V7 图片为 4000 张。自主采集部分来自 10 张独立拍摄的母图。每张母图先统一尺寸，再生成 99 个颜色、亮度、尺度、平移或轻微旋转版本，最终得到 100 个版本。因此，这里的 1000 张指实拍来源及其衍生素材，并不代表进行了 1000 次独立拍摄。母图中出现的类别为 mouse、keyboard 和 laptop，cup 与 headphones 样本由公开数据补充。公开图片继续保留原始来源 ID 和划分信息。

若同一母图的不同版本同时进入训练集和测试集，测试结果会偏高。为避免这种情况，10 张母图按 8:1:1 分配，某一母图生成的全部版本只属于一个集合。数据生成后逐项检查图片能否读取、图片与标签是否配对、类别编号和坐标是否有效，同时核对重复来源与跨集合母图。图2给出了部分自主采集素材及其标注。图中变化由设定的图像变换产生，类别和边界框关系保持不变。

表II统计了各类目标框。laptop 有 2549 个，占 29.96%，数量最多。headphones 只有 871 个，占 10.24%。由于类别数量存在差异，模型评价不能只看总体均值，还需要比较各类别的 Precision、Recall 和 AP。

V. 实验设置与评价方法

A. 实验环境

训练和软件测试均在 macOS 上完成，处理器为 Apple M4，计算后端使用 MPS。Python、PyTorch 和 Ultralytics 的版本依次为 3.12.4、2.13.0 和 8.4.133。训练与测试的输入尺寸均为 640。第 12 轮保存的权重在验证集上取得 0.7407 的 Precision、0.7512 的 Recall、0.7334 的 mAP@0.5 和 0.4621 的 mAP@0.5:0.95，因此作为后续测试模型。

B. 评价指标与测试协议

设真正例、假正例和假负例分别为 TP、FP 和 FN，则 Precision 与 Recall 定义为

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}. \tag{1}$$

预测框与真实框的交并比为

$$\mathrm{IoU} = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}, \tag{2}$$

其中 B_p 和 B_g 分别为预测框与真实框。mAP@0.5 在 IoU 阈值 0.5 下计算各类别 AP 的均值，mAP@0.5:0.95 则在 0.5 至 0.95、步长 0.05 的多个阈值上平均，因而对边界框定位更严格 [7], [8]。

模型采用三种测试方式。在 650 张测试图像上统计总体和逐类指标，并生成 PR 曲线与混淆矩阵。另从已保存的单目标图片中为每个类别选取 4 张，共 20 张。只要指定类别被检出，即记为该图识别正确。视频速度使用同一段固定视频测量。程序先运行 20 帧预热，再对后续 130 帧计时，耗时包括读取、预处理、推理、后处理和绘制，而不是只统计网络前向时间。

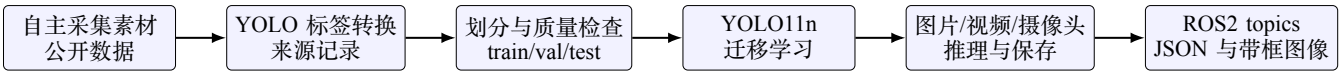


Fig. 1. 桌面目标检测系统的端到端处理流程。

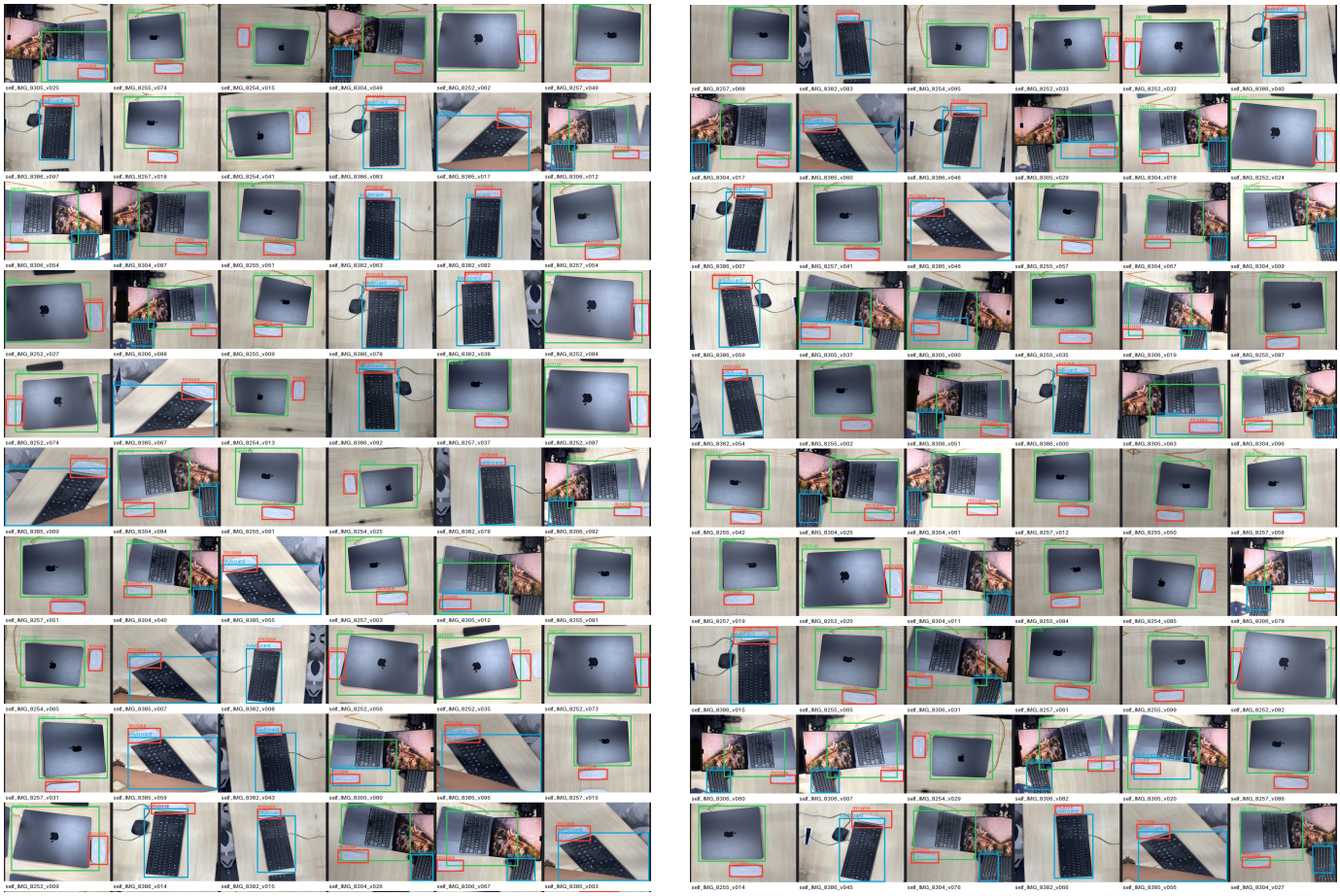


Fig. 2. 自主采集来源素材的增强与标注样例。

VI. 实验结果与分析

A. 测试集总体结果

650 张测试图像的结果列于表III。总体 Precision 为 0.8385，高于 0.7697 的 Recall，测试中的漏检问题比误检更突出。mAP@0.5 为 0.8386。当 IoU 阈值从 0.5 提高并在 0.5 至 0.95 范围内取平均时，mAP@0.5:0.95 降为 0.5735。二者相差 0.2651，说明部分预测框能够给出正确类别，但与真实框的贴合程度还不够稳定。

mouse 的结果最好，mAP@0.5 和 Recall 分别达到 0.9501 和 0.9101。keyboard 的 mAP@0.5 为 0.9012，说明笔记本键盘与独立键盘同时存在的标注方式没有妨碍模型识别该类。laptop 的 Precision 为 0.9121，但 Recall 只有 0.7529，mAP@0.5:0.95 也降到 0.4599。结合保存的检测图可以看到，笔记本受到遮挡、处于不同开合角度或靠近画面边缘时，整机框更容易漏检或偏移。

headphones 的 Precision、Recall 和 mAP@0.5 分别为 0.6041、0.5978 和 0.6203，是五类中最低的。该类样本数最少，同时存在展开、折叠和局部遮挡等外观差异。现有结果只能说明样本数量与检测表现同时偏低，尚不能据此断定二者存在直接因果关系。不过，下一轮采集可优先补充不同姿态和尺度的耳机图片。图3中的归一化混淆矩阵进一步显示了各类别的漏检和混淆情况。

TABLE III
测试集总体与逐类检测指标

类别	Precision	Recall	mAP@0.5	mAP@0.5:0.95
mouse	0.9325	0.9101	0.9501	0.7546
keyboard	0.8346	0.8469	0.9012	0.6361
laptop	0.9121	0.7529	0.8640	0.4599
cup	0.9091	0.7409	0.8571	0.6751
headphones	0.6041	0.5978	0.6203	0.3419
总体	0.8385	0.7697	0.8386	0.5735

B. 训练过程与 20 样本测试

图4记录了 12 轮训练过程。训练损失整体下降，验证集 mAP 在后几轮继续提高，最佳权重出现在第 12 轮，也就

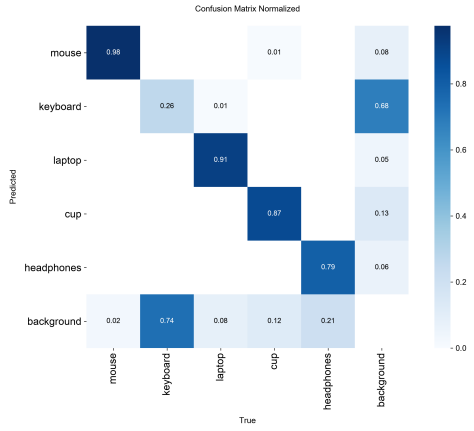


Fig. 3. 650 张测试图像上的归一化混淆矩阵。

TABLE IV
主要实验结果汇总

测试项目	结果
测试集规模	650 张，1107 个目标框
总体 mAP@0.5 / mAP@0.5:0.95	0.8386 / 0.5735
20 样本正确率	19/20，95%
平均推理耗时	10.087 ms
平均端到端速度	60.409 FPS
最低瞬时速度	34.733 FPS

是本次训练的最后一轮。验证损失存在波动，但没有伴随验证指标连续下降。由于初始权重已在相同五类目标上训练过，本次实验采用了较短的微调周期。

20 张样本中有 19 张识别正确，正确率为 95%，超过实验要求的 80%。错误样本属于 laptop，模型没有输出达到置信度阈值的 laptop 框。这一现象与测试集指标相符：laptop 的 Precision 较高而 Recall 偏低，问题主要表现为部分画面不出框。该错误图片已单独保存，之后可以按背景、目标尺度和遮挡情况复查。

C. 连续视频性能

固定视频共记录 130 个计时帧。模型推理平均耗时 10.087 ms，包含视频读取和结果绘制在内的平均耗时为 16.554 ms，即 60.409 FPS。测试中最低瞬时速度为 34.733 FPS，最大单帧模型推理耗时为 13.450 ms。网络前向之外的时间主要花在视频读取、图像预处理、边界框绘制和结果整理上。在本机测试条件下，完整处理速度始终高于 5 FPS。主要数值汇总于表IV。

VII. ROS2 通信与边缘部署设计

A. 检测节点与主题接口

检测程序被封装为独立的 ROS2 节点。使用 USB 摄像头时，节点直接读取视频帧。接入已有相机驱动时，则订阅 `/camera/image_raw`。每帧推理完成后，节点发布两个主题。`/object_detection/result` 以 JSON 字符串记录时间戳、帧号、FPS、类别编号、类别名称、置信度和边界框。`/object_detection/image` 的类型为 `sensor_msgs/Image`，内容是绘制检测框后的图像。

数值结果和可视化图像分开发布后，控制程序只需接收 JSON，不必处理完整图像。调试时再订阅带框画面。ROS2 允许多个订阅者同时接收同一 topic[9]。运行过程中可用 `ros2 topic echo` 查看字段，用 `ros2 topic hz` 统计接收频率，并在 `rqt_image_view` 中查看检测画面。

B. 边缘推理流程

部署使用训练生成的 `best.pt`。该权重可由 PyTorch 直接加载，也可先导出为 ONNX，再由 TensorRT 构建序列化 engine。TensorRT 会根据目标 GPU 选择各层实现，运行时加载 engine 执行推理 [10]。engine 与操作系统、CPU 架构、GPU 型号和 TensorRT 版本有关，因此应在目标设备上生成，同时固定输入尺寸和计算精度。

部署配置沿用训练时的类别顺序和 640 像素输入。切换为 FP16 时，置信度阈值和 NMS 参数保持不变。测速前先处理若干预热帧，避免首次加载与内核初始化影响结果，再统计固定帧数的端到端 FPS。若需要降低计算量，可比较 640、512 和 416 三种输入尺寸。每次改变尺寸后，都要重新执行 20 张样本测试，确认提速没有明显降低识别正确率。

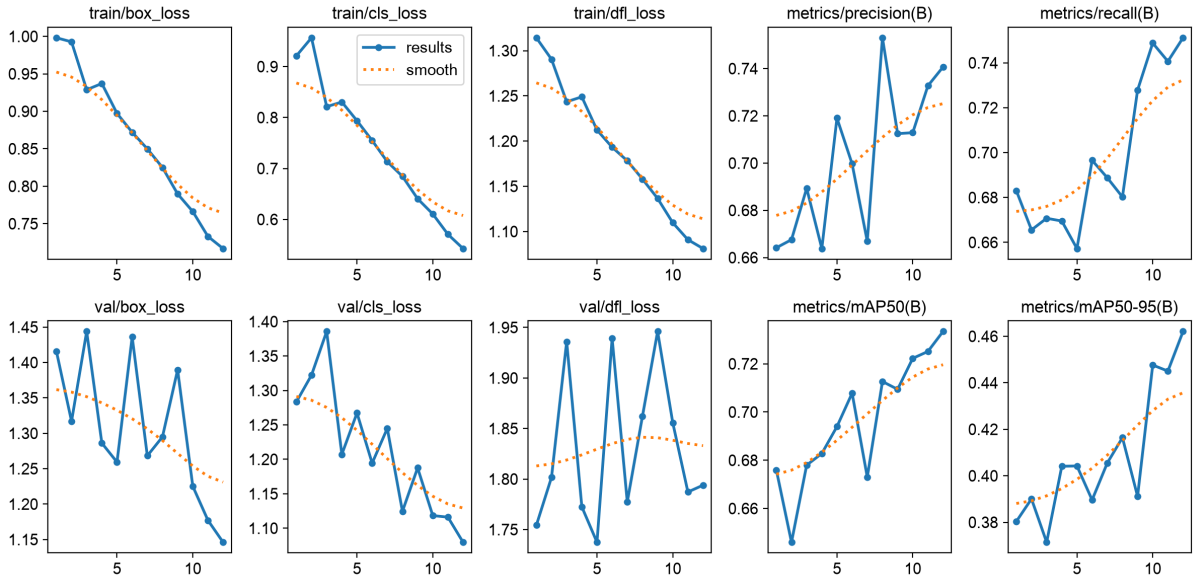


Fig. 4. YOLO11n 在 12 轮迁移学习中的损失和验证指标曲线。

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

[2] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Computer Vision – ECCV 2016*, 2016, pp. 21–37.

[3] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2980–2988.

[4] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.

[5] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024, version 11.0.0, accessed: 2026-09-03. [Online]. Available: https://github.com/ultralytics/ultralytics

[6] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Malloci, A. Kolesnikov, T. Duerig, and V. Ferrari, "The open images dataset v4: Unified image classification, object detection, and visual relationship detection at scale," *International Journal of Computer Vision*, vol. 128, no. 7, pp. 1956–1981, 2020.

[7] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, 2014, pp. 740–755.

[8] Ultralytics, "Model validation with ultralytics YOLO," https://docs.ultralytics.com/modes/val/, 2026, accessed: 2026-09-03.

[9] Open Robotics, "ROS 2 documentation: Topics," https://docs.ros.org/en/ros2_documentation/kilted/Concepts/Basic/About-Topics.html, 2026, accessed: 2026-09-03.

[10] NVIDIA Corporation, "How TensorRT works," https://docs.nvidia.com/deeplearning/tensorrt/latest/architecture/how-trt-works.html, 2026, accessed: 2026-09-03.